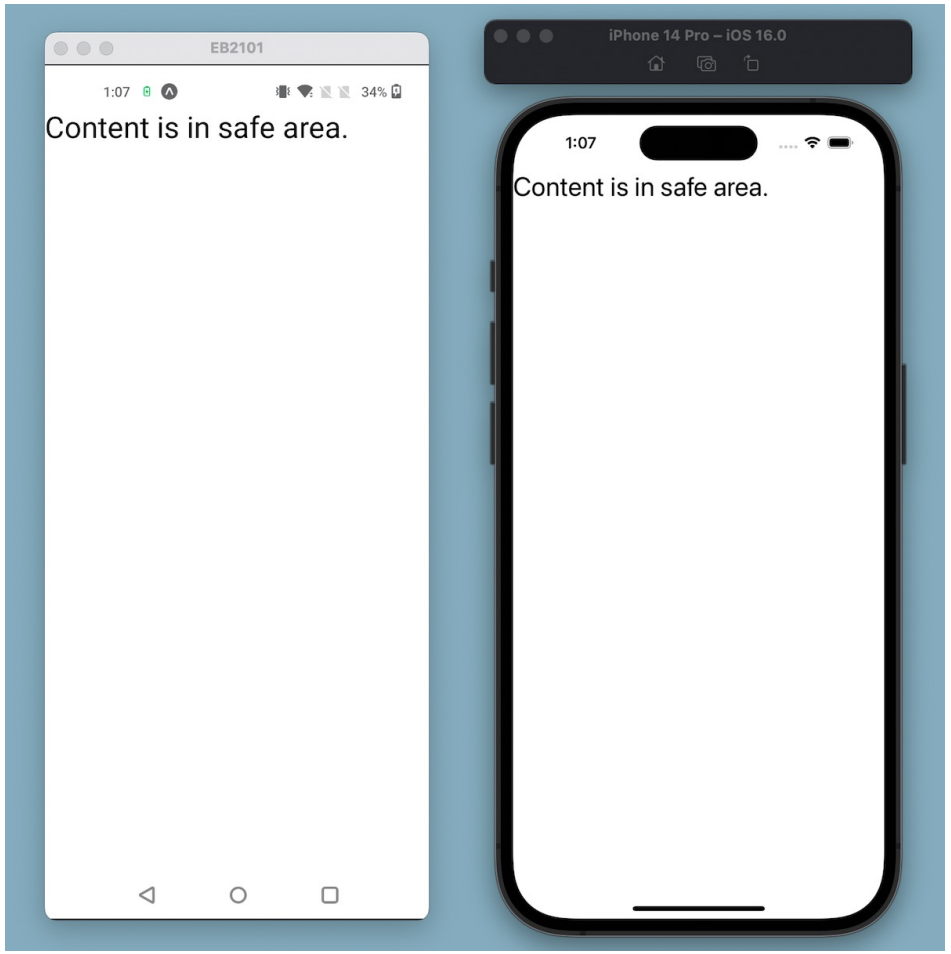
RN Design

Safe areas

Creating a safe area ensures your app screen's content is positioned correctly. This means it doesn't get overlapped by notches, status bars, home indicators, and other interface elements that are part of the device's physical hardware or are controlled by the operating system. When the content gets overlapped, it gets concealed by these interface elements.

Use react-native-safe-area-context library

npx expo install react-native-safe-area-context



```tsx
import { Text } from 'react-native';
import { SafeAreaView } from 'react-native-safe-area-context';

export default function App() {
  return (
    <SafeAreaView style={{ flex: 1 }}>
      <Text>Content is in safe area.</Text>
    </SafeAreaView>
  );
}
```

Usage: SafeAreaView

Usage: useSafeAreaInsets

```tsx
import { Text, View } from 'react-native';
import { SafeAreaProvider, useSafeAreaInsets } from
'react-native-safe-area-context';

function HomeScreen() {
  const insets = useSafeAreaInsets();
  return (
    <View style={{ flex: 1, paddingTop: insets.top }}>
      <Text style={{ fontSize: 28 }}>Content is in safe area.</Text>
    </View>
  );
}

export default function App() {
  return (
    <SafeAreaProvider>
      <HomeScreen />
    </SafeAreaProvider>
  );
}
```

Style

With React Native, you style your application using JavaScript. All of the core components accept a prop named *style*. The *style* names and *values* usually match how CSS works on the web, except names are written using camel casing, e.g. *backgroundColor* rather than *background-color*.

```
import React from 'react';
import { StyleSheet, Text, View } from 'react-native';

const Main = () => {
  return(
    <View style={styles.container}>
      <Text style={styles.red}>just red</Text>
      <Text style={styles.bigblue}>just BigBlue</Text>
    </View>
  );
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: '#fff',
    marginTop: 50
  },
  red: {
    color: 'red'
  },
  bigblue: {
    color: 'blue',
    fontWeight: 'bold',
    fontSize: 30
  }
});

export default Main
```

Height and Width

- A component's height and width determine its size on the screen.
- The general way to set the dimensions of a component is by adding a fixed width and height to style. All dimensions in React Native are unitless.

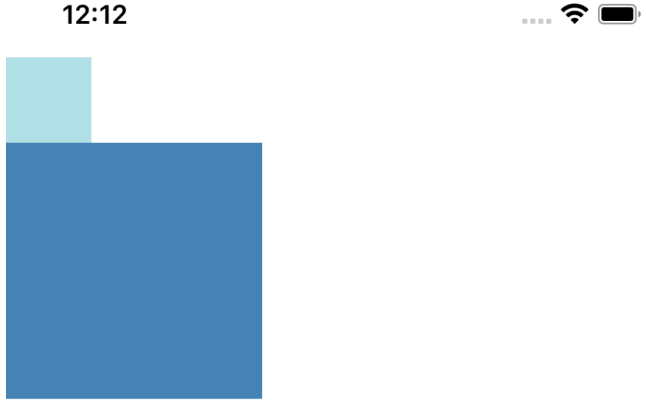
- Fixed Dimensions
- Flex Dimensions
- Percentage Dimensions

```jsx
import React from 'react';
import { StyleSheet, Text, View } from 'react-native';

const FixedDimensions = () => {
  return(
    <View style={styles.container}>
      <View style={styles.box1}></View>
      <View style={styles.box2}></View>
    </View>
  );
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: '#fff',
    marginTop: 50
  },
  box1: {
    width: 50,
    height: 50,
    backgroundColor: 'powderblue'
  },
  box2: {
    width: 150,
    height: 150,
    backgroundColor: 'steelblue'
  }
});

export default FixedDimensions
```

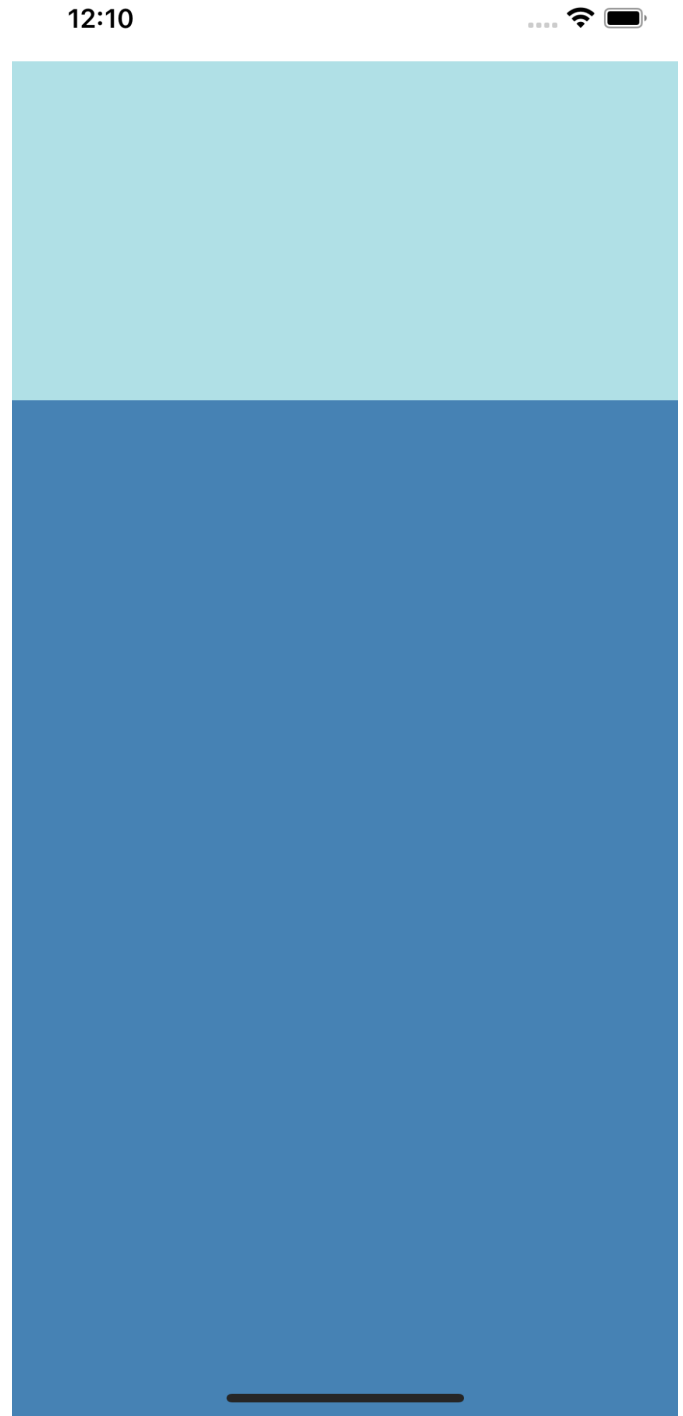


```jsx
import React from 'react';
import { StyleSheet, Text, View } from 'react-native';

const FlexDimensions = () => {
  return(
    <View style={styles.container}>
      <View style={styles.box1}></View>
      <View style={styles.box2}></View>
    </View>
  );
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: '#fff',
    marginTop: 50
  },
  box1: {
    flex: 1,
    backgroundColor: 'powderblue'
  },
  box2: {
    flex: 3,
    backgroundColor: 'steelblue'
  }
});

export default FlexDimensions
```



```jsx
import React from 'react';
import { StyleSheet, Text, View } from 'react-native';

const PercentageDimensions = () => {
  return(
    <View style={styles.container}>
      <View style={styles.box1}></View>
      <View style={styles.box2}></View>
    </View>
  );
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: '#fff',
    marginTop: 50
  },
  box1: {
    height: '15%',
    backgroundColor: 'powderblue'
  },
  box2: {
    height: '50%',
    width: '33%',
    backgroundColor: 'steelblue'
  }
});

export default PercentageDimensions
```

Layout with Flexbox

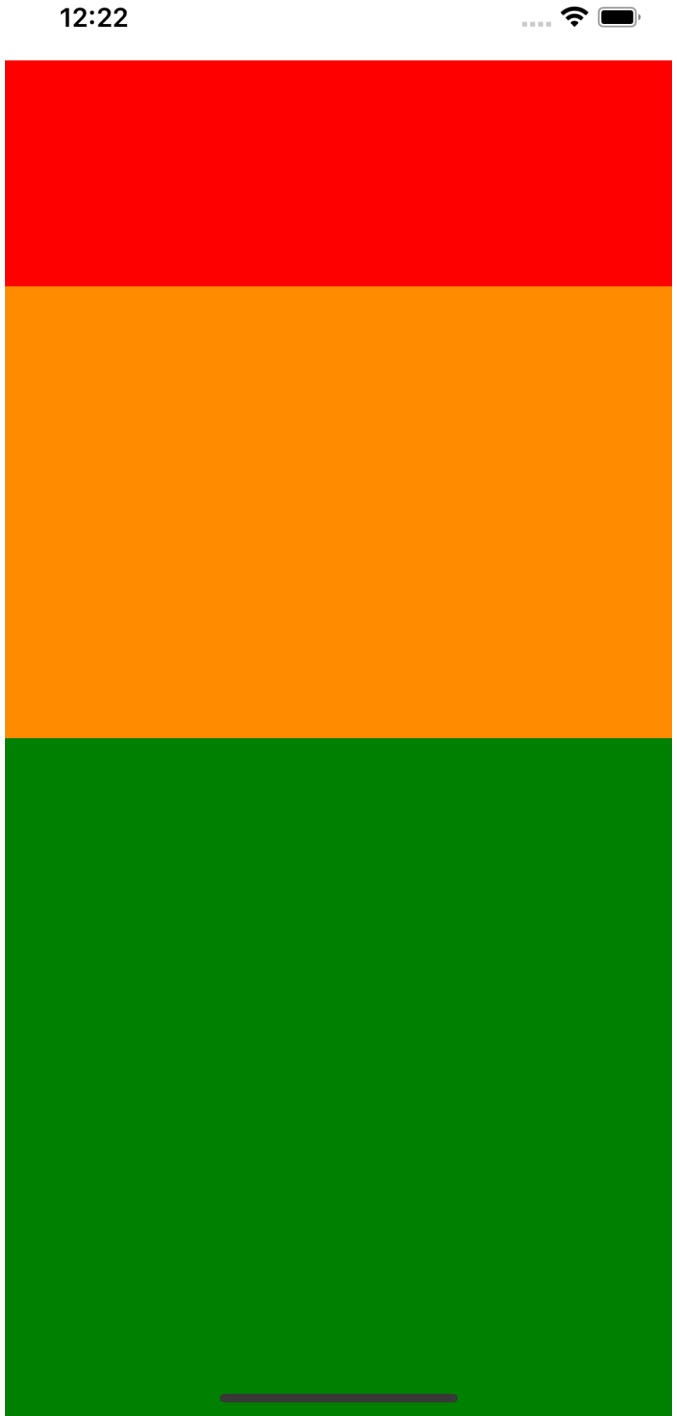
- A component can specify the layout of its children using the Flexbox algorithm. Flexbox is designed to provide a consistent layout on different screen sizes.
- You will normally use a combination of flexDirection, alignItems, and justifyContent to achieve the right layout.

```jsx
import React from 'react';
import { StyleSheet, Text, View } from 'react-native';

const Flex = () => {
  return(
    <View style={styles.container}>
      <View style={styles.box1}></View>
      <View style={styles.box2}></View>
      <View style={styles.box3}></View>
    </View>
  );
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    flexDirection: 'column',
    backgroundColor: '#fff',
    marginTop: 50
  },
  box1: {
    flex: 1,
    backgroundColor: 'red'
  },
  box2: {
    flex: 2,
    backgroundColor: 'darkorange'
  },
  box3: {
    flex: 3,
    backgroundColor: 'green'
  }
});

export default Flex
```



Color Reference

- Named colors implementation follows the *CSS3/SVG specification*
- *https://www.w3.org/TR/css-color-3/#svg-color*

Flex Direction

- FlexDirection determines the direction children should render. You can code with values: column, row, column-reverse, and row-reverse, but the default is column.

```jsx
import React from 'react';
import { StyleSheet, Text, View } from 'react-native';

const FlexDirection = () => {
  return(
    <View style={styles.container}>
        <View style={styles.box}></View>
        <View style={styles.box}></View>
        <View style={styles.box}></View>
    </View>
  );
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: '#fff',
    alignItems: 'center',
    justifyContent: 'center',
    flexDirection: 'row'
  },
  box: {
    height: 100,
    width: 100,
    margin: 4,
    backgroundColor: 'powderblue'
  },

});

export default FlexDirection
```

Justify Content

- justifyContent determines content along the primary axis that is affected by the flexDirection. If flexDirection is column, then it's vertical. If it's classified as row, it's horizontal.
- As a reminder, here are the available values: flex-start, flex-end, center, space-between, space-around, and space-evenly.

```jsx
import React from 'react';
import { StyleSheet, Text, View } from 'react-native';

const FlexDirection = () => {
  return(
    <View style={styles.container}>
      <View style={styles.box}></View>
      <View style={styles.box}></View>
      <View style={styles.box}></View>
    </View>
  );
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: '#fff',
    justifyContent: 'center',
    flexDirection: 'column'
  },
  box: {
    height: 100,
    width: 100,
    margin: 4,
    backgroundColor: 'powderblue'
  },

});

export default FlexDirection
```

Align Items

- alignItems determines how an item should be rendered along the secondary axis, which is determined by the flexDirection property. This is the inverse of justifyContent. So, if justifyContent is handling the vertical alignment, then alignItems is handling the horizontal alignment.
- Available values: flex-start, flex-end, center, and baseline.

```jsx
import React from 'react';
import { StyleSheet, Text, View } from 'react-native';

const FlexDirection = () => {
  return(
    <View style={styles.container}>
        <View style={styles.box}></View>
        <View style={styles.box}></View>
        <View style={styles.box}></View>
    </View>
  );
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    flexDirection: 'column',
    backgroundColor: '#fff',
    justifyContent: 'center',
    alignItems: 'flex-end'
  },
  box: {
    height: 100,
    width: 100,
    margin: 4,
    backgroundColor: 'powderblue'
  },

});

export default FlexDirection
```

Align Self

- alignSelf determines how a child should align itself, and it overrides alignItems. The available values for alignSelf are flex-start, flex-end, center, baseline.

```jsx
import React from 'react';
import { StyleSheet, Text, View } from 'react-native';

const FlexDirection = () => {
  return(
    <View style={styles.container}>
      <View style={styles.box}></View>
      <View style={[styles.box, {alignSelf: 'flex-end'}]}></View>
      <View style={[styles.box, {alignSelf: 'flex-start'}]}></View>
    </View>
  );
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    flexDirection: 'column',
    backgroundColor: '#fff',
    justifyContent: 'center',
    alignItems: 'center'
  },
  box: {
    height: 100,
    width: 100,
    margin: 4,
    backgroundColor: 'powderblue'
  },

});

export default FlexDirection
```